HACKSAGON 2025

Indian Institute of Information Technology, Gwalior

Smart Resume Builder AI

An AI-Powered Career Toolkit for Students and Fresh Graduates

---

Team Name: The Silicon Savants

Members: Gyan Chandra

Dristi Singh

Technology Stack: Node.js, React, MongoDB, OpenAI API

A technical report submitted for Hacksagon 2025 documenting the design, architecture, and implementation of the Smart Resume Builder AI platform.

1 Problem Statement

Students and fresh graduates often struggle to present their skills effectively. This structural barrier leads to rejection before a human recruiter even reviews their application. The core challenges include:

- **Writing Gap:** Technical proficiency does not automatically translate into professional resume writing. Students frequently use passive language and weak descriptions.
- **ATS Blindness:** Applicant Tracking Systems (ATS) filter a vast majority of resumes. Applicants often lack the knowledge of which keywords are required for their target roles.
- **Lack of Guidance:** Career counselling and professional resume review services are expensive and largely inaccessible.

Existing tools primarily solve formatting issues but fail to address content quality, ATS compatibility, or interview preparation. Smart Resume Builder AI was developed to address these critical gaps in a unified platform.

2 System Architecture

The platform employs a robust three-tier architecture: a React Single Page Application (SPA) for the frontend, an Express REST API backend, and a MongoDB database. All third-party AI requests are securely executed server-side.

2.1 Architecture Diagram

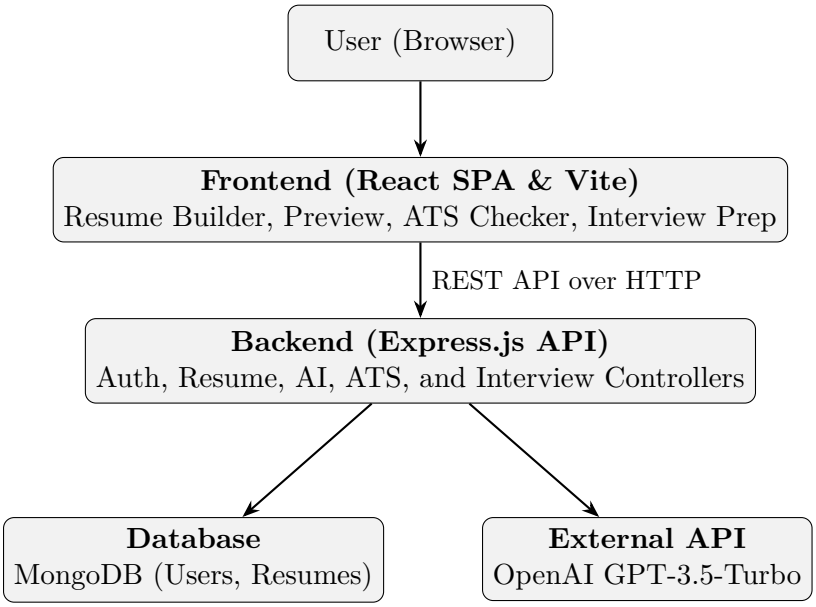

Figure 1: Three-tier system architecture

2.2 Request Lifecycle

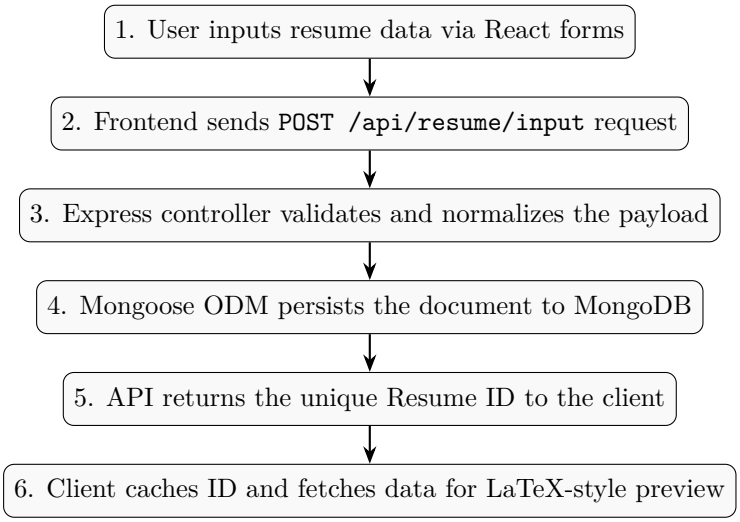

Figure 2: Request lifecycle for resume creation

3 Technology Stack

Table 1: Project Technology Stack

Layer	Technology	Role
Frontend	React 18, Vite	SPA framework and development build tool
	Tailwind CSS v4	Utility-first CSS for UI components
	React Router v6	Client-side navigation
Backend	Node.js 22	JavaScript runtime environment
	Express 5	RESTful HTTP server and route handling
	jsonwebtoken	Stateless JWT authentication
	bcryptjs	Secure password hashing
Database	MongoDB	Document-oriented storage
	Mongoose 8	Data modeling and schema validation
AI	OpenAI API	Content generation via GPT-3.5-Turbo

4 Database Design

The application maintains two primary collections: `users` for authentication credentials, and `resumes` for structured career documents.

```
const resumeSchema = new mongoose.Schema({
  name: String,
  email: String,
```

```
  phone: String,
  address: String,
  socialLinks: { linkedin: String, github: String, portfolio: String },
  roleAppliedFor: String,
  skills: [String],
  objective: String,
  education: [{ degree: String, college: String, year: Number }],
  experience: [{ company: String, role: String, duration: String,
     description: String }],
  projects: [{ title: String, techStack: String, description: String,
     githubLink: String }],
  certifications: [{ name: String, issuer: String, date: String }]
}, { timestamps: true });
```

Listing 1: Resume Mongoose Schema Definition

5 Key Features

5.1 Dynamic Resume Builder

The platform provides a guided form with dynamic add/remove operations for nested fields such as Education and Experience. All states are managed locally via React, and upon submission, the payload is normalized and persisted to MongoDB.

5.2 AI Professional Summary

By transmitting the user's role, skills, and base objective to GPT-3.5-Turbo, the application generates a professional summary, three achievement-oriented bullet points, and an improved career objective. A rule-based fallback ensures continuity if the API is unreachable.

5.3 ATS Score Checker

The system analyzes the candidate's resume against a target job description, returning an ATS score (0–100), key strengths, specific weaknesses, missing keywords, and actionable recommendations.

5.4 Cover Letter Generator

The application automatically synthesizes a highly targeted, 300-word cover letter tailored to a specified role and company, utilizing the user's existing resume data for context.

5.5 Interview Question Generator

Leveraging the resume context, the system generates a personalized set of 10 interview questions (behavioral, technical, and situational), complete with difficulty ratings and recommended answering strategies.

5.6 LaTeX-Quality Resume Preview

The frontend renders the data using precise inline CSS to replicate a professional LaTeX document. Features include Times New Roman typography, small-caps section headers, accurate margins, and flawless PDF export.

6 API Reference

All REST endpoints follow a uniform JSON response envelope: `{ success, message, data }`.

Table 2: REST API Endpoints

Method	Endpoint	Description
POST	`/api/auth/signup`	Register a new user
POST	`/api/auth/login`	Authenticate and return JWT
POST	`/api/resume/input`	Create and persist resume
GET	`/api/resume/:id`	Fetch resume by ID
POST	`/api/resume/generateSummary`	AI: Generate professional summary
POST	`/api/resume/generateCoverLetter`	AI: Generate cover letter
POST	`/api/resume/ats-check`	AI: ATS scoring and keyword analysis
POST	`/api/interview/questions`	AI: Generate interview questions

7 Implementation Challenges

Express 5 Wildcard Routing: Express 5 introduced strict path parsing via `path-to-regexp` v8, which rejects legacy wildcard syntax. The SPA fallback mechanism was rewritten as an `app.use()` middleware to conditionally serve `index.html` without triggering parser errors.

OpenAI Lazy Initialization: Immediate initialization of the OpenAI client caused application crashes when environment variables were absent. This was resolved by deferring initialization to within the request handlers, allowing standard routes to function seamlessly without an API key.

Development CORS Proxying: To streamline local development, Vite was configured to transparently proxy `/api/*` requests to the Express backend, negating the need for complex CORS configurations and hardcoded URLs in the React client.

8 Conclusion and Future Scope

Smart Resume Builder AI successfully integrates generative AI within a modern full-stack web application to solve tangible problems for job seekers. Developed for Hacksagon 2025,

the platform bridges the gap between technical capability and professional presentation.

Future iterations of the platform aim to implement server-side PDF generation via Puppeteer, automated resume parsing for existing documents, and a collaborative peer-review system to foster community-driven career development.

The Silicon Savants
Gyan Chandra · Dristi Singh
Hacksagon 2025, IIIT Gwalior